

Recursion Tree Analysis for Three-Way Division

Solving $T(n) = 3T(n/3) + n$ using the Recursion Tree Method, verified against the Master Theorem

PRESENTED BY

Y. Greeshma Reddy (192424146)

GUIDED BY

Dr. A. Sairam , Dept. of CSE

Problem Statement

- Many divide-and-conquer algorithms split a problem of size n into 3 equal sub-problems of size $n/3$, then spend $O(n)$ linear work combining the results.
- This behavior is captured by the recurrence $T(n) = 3T(n/3) + n$ — but its growth rate is not obvious just by looking at the equation.
- Naively unrolling the recursion by hand is tedious and error-prone once the number of levels grows large.
- A systematic, visual technique is needed to compute the exact per-level cost and the total running time with confidence.

GOAL

Determine the asymptotic time complexity of

$$T(n) = 3T(n/3) + n$$

using the recursion tree method, then cross-check the result against the Master Theorem.

$$a = 3 \quad b = 3 \quad f(n) = n$$

Algorithm Overview

The Recursion Tree Method

A recursion tree visualizes every recursive call as a node. Each node's label is the non-recursive (combine) cost incurred at that call. Summing the labels level-by-level, then summing across all levels, gives the exact total cost until the base case.

1

Build the tree

2

Label each node

Write the non-recursive work done at that call — here, the size of the sub-problem itself.

3

Sum each level

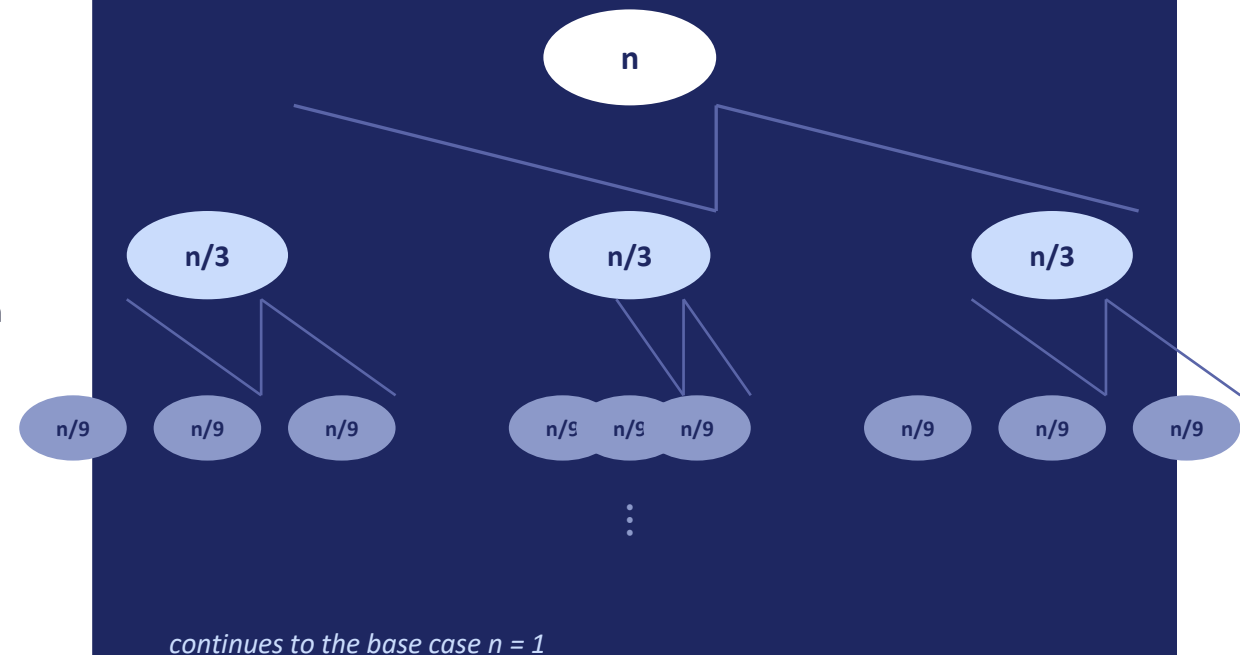
Add the labels across a level to get that level's total cost.

4

Sum all levels

Add every level's cost, from the root to the base cases, to get $T(n)$.

THREE-WAY BRANCHING



Algorithm / Pseudocode

Generic Divide-and-Conquer Procedure

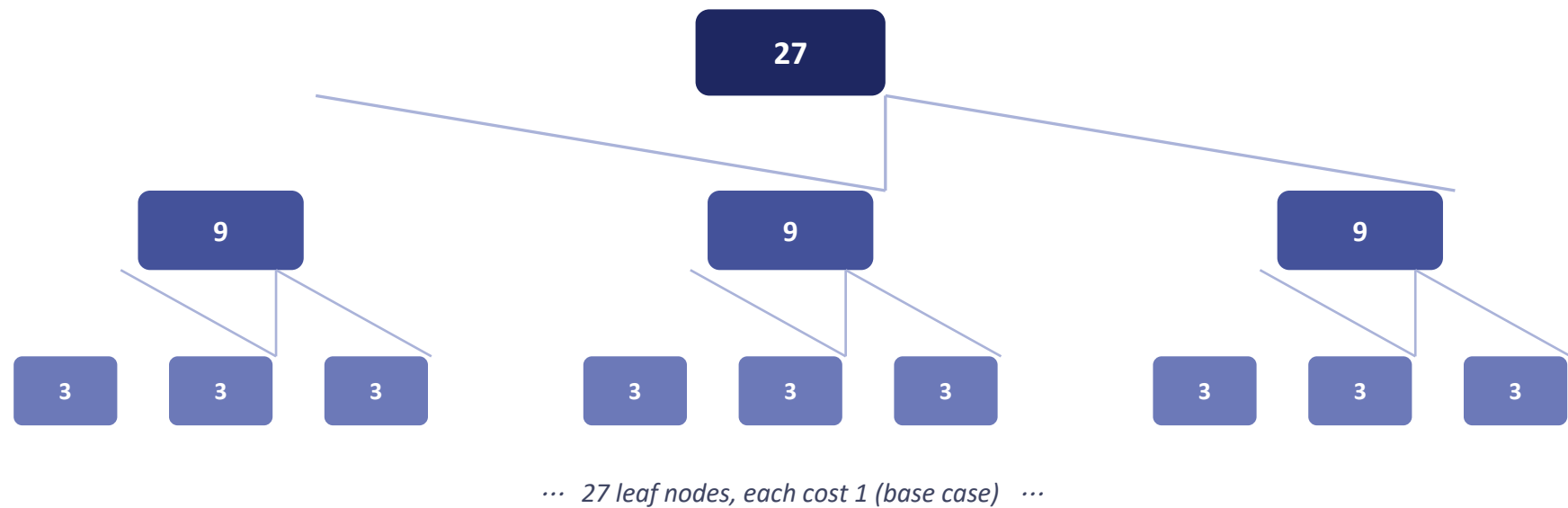
```
SOLVE(A, n):  
    if n <= 1:  
        return O(1)                // base case  
  
    split A into A1, A2, A3        // 3 parts, size  
    n/3  
  
    R1 = SOLVE(A1, n/3)  
    R2 = SOLVE(A2, n/3)  
    R3 = SOLVE(A3, n/3)  
  
    result = COMBINE(R1, R2, R3)  // O(n) work  
    return result
```

Recursion-Tree Construction Steps

```
BUILD_TREE(n):  
    root <- node(cost = n)  
    queue <- [ (root, n) ]  
    level <- 0  
    while any node size > 1:  
        for each (node, size) in queue:  
            if size == 1: mark as leaf, cost = O(1)  
            else: create 3 children,  
                    each with size = size/3,  
                    each with cost = size/3  
        level <- level + 1  
    return root, total levels
```

Working Example — Dry Run

Take $n = 27 = 3^3$ so the tree divides evenly down to the base case.



Level	Nodes	Size per node	Cost per level
0	1	27	$1 \times 27 = 27$
1	3	9	$3 \times 9 = 27$
2	9	3	$9 \times 3 = 27$
3 (leaves)	27	1	$27 \times 1 = 27$

TOTAL COST

4 levels $\times$ 27 = 108

= $n \cdot (\log_3 n + 1) \Rightarrow O(n \log n)$

Complexity Analysis

General Derivation

Cost per level

Every level sums to n , since $3 \text{ nodes} \times (n/3) = n$ at level 1, $9 \times (n/9) = n$ at level 2, and so on.

Height of the tree

The size shrinks by a factor of 3 each level, reaching the base case (size 1) after $\log_3 n$ levels.

Total cost

$T(n) = (\text{cost per level}) \times (\text{number of levels}) = n \times \log_3 n = \Theta(n \log n)$.

MASTER THEOREM CROSS-CHECK

$$T(n) = aT(n/b) + f(n)$$

$$a = 3 \quad b = 3 \quad f(n) = n$$

$$n^{(\log_b a)} = n^{(\log_3 3)} = n^1 = n$$

$$\text{Compare with } f(n) = n \rightarrow f(n) = \Theta(n^{(\log_b a)})$$

Case 2 of the Master Theorem applies

$$T(n) = \Theta(n \log n)$$

Identical to the recursion-tree result — the two methods agree.

Comparative Analysis

How the Recursion Tree Method compares to other techniques for solving $T(n) = 3T(n/3) + n$

Method	Approach	Effort	Best used when
Recursion Tree	Visualizes every call, sums cost level-by-level	Moderate — clear but can be large	You want an intuitive, verifiable derivation
Master Theorem	Plug a, b, f(n) into 3 standard cases	Low — fast once conditions are checked	The recurrence fits a, b, f(n) form directly
Substitution Method	Guess a bound, prove it by induction	High — needs a correct guess up front	Recurrence is irregular or non-standard
Iteration Method	Expand the recurrence algebraically, step by step	High — tedious for many levels	Small recurrences or verifying a guess



Why the recursion tree here: with a clean 3-way split and linear combine cost, the tree stays perfectly balanced, making it easy to spot that every level costs exactly n — a pattern the Master Theorem then confirms in one step.

Applications

1

3-Way Merge Sort

Splits an array into three parts instead of two, merging them with an $O(n)$ three-way merge step.

2

Ternary Search Trees

Balanced 3-way branching supports fast prefix search and string-key retrieval.

3

Parallel / Distributed D&C

Workloads partitioned across 3 workers per stage, common in map-reduce style pipelines.

4

Multi-way Partition Sorting

Algorithms like 3-way quicksort variants that split around multiple pivots.

5

Database Index Structures

B-tree-like structures with branching factor 3 use the same balanced-cost reasoning.

6

Hierarchical Data Aggregation

Sensor or log aggregation trees that fan out into 3 regional nodes per level.

Results & Discussion

Growth of $T(n) = \Theta(n \log n)$

n	$\log_3 n$	$T(n) \approx n \cdot \log_3 n$
27	3	81
81	4	324
243	5	1,215
729	6	4,374

As n grows, the total cost scales just slightly faster than linear — the $\log_3 n$ factor grows very slowly, so the algorithm remains highly efficient in practice.

KEY OBSERVATIONS

- The recursion tree gives the same $\Theta(n \log n)$ result as the Master Theorem, confirming correctness through two independent methods.
- The balanced 3-way split keeps every level's cost identical ($= n$), which is what makes this recurrence a textbook Case-2 example.
- The recursion tree is especially valuable for teaching and debugging: it shows visually why the answer is $n \log n$, not just that it is.
- Limitation: for uneven splits or non-uniform $f(n)$, the tree becomes irregular and the level sums must be bounded rather than computed exactly.

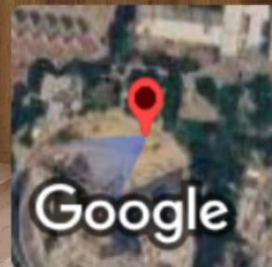
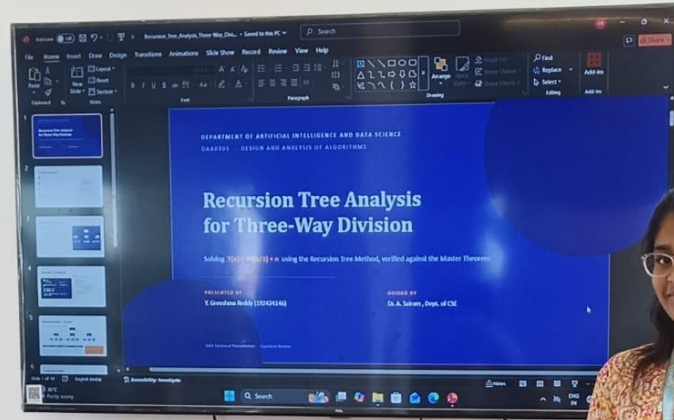
CONCLUSION

$T(n) = 3T(n/3) + n$ resolves to $\Theta(n \log n)$

- ✓ The recursion tree method shows each of the $\log_3 n$ levels costs exactly n , giving a total of $n \log_3 n = \Theta(n \log n)$.
- ✓ Applying the Master Theorem (Case 2, since $f(n) = \Theta(n^{\log_b a})$) independently confirms the identical bound.
- ✓ This confirms the recursion tree as a reliable, visual way to derive — and sanity-check — complexity bounds for balanced divide-and-conquer recurrences.

REFERENCES

-
- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
 - [2] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
 - [3] Horowitz, E., Sahni, S., & Rajasekaran, S. (2019). Fundamentals of Computer Algorithms (2nd ed.). Universities Press.



Mevalurkuppam, Tamil Nadu,
India 🇮🇳

22h8+7xq, Mevalurkuppam, Tamil Nadu 602105,
India

Lat 13.027104° Long 80.016912°

Wednesday, 22/07/2026 02:11 PM GMT +05:30